ARCHITECTURE AND CODE REFERENCE · GENERATED 2026-10-27 FROM THE CODE

Expense Auto-Approver

An approver for Finance Ops that decides approve, reject or flag for each expense, drafts a follow-up question for flags, POSTs every decision downstream, and serves an employee portal and a Finance dashboard.

FastAPI · SQLite (WAL) · React 18 + Vite + Tailwind · Claude (extraction, questions, guard fallback) · OpenAI Decisions API (guard)
Policy version 2026-10-27.5 · prompt version v3 · last test result: 156 passed · eval 38/38 on labelled data · mock suite 99/100

Contents

1 Overview and design rules
2 Running it
3 Repository layout
4 Decision flow
5 Backend reference: every module and function
6 Policy rules reference
7 REST and SSE API
8 Data model and what is POSTed
9 Frontend reference
10 Scripts and prompts
11 Tests
12 Safety boundaries
13 LLM providers and fallback
14 Failure modes and concurrency
15 Decisions (ADRs)
16 Known gaps and planned work

1. Overview and design rules

Expenses arrive from a batch file (`batch.py`) or the portal, and both go through one function, `pipeline.process`. Each is screened by an input guard, normalised, has facts extracted by an LLM, and is decided by a pure policy engine; only flags get an LLM-drafted question. Every decision is an append-only revision. It is delivered through a retrying outbox and pushed to the UI over SSE, and a follow-up answer triggers a full re-decision.

Approve
Within every cap and rule, all checks finished, $1,000 or less, and nothing suspicious.

Reject
Over a meal or hotel cap, a weekend meal or entertainment with no client, or a confident manipulation attempt.

Flag + question
Missing or unsure data: a receipt over $25, an approval code, a non-USD amount, a duplicate, a split, a stale date, a failed check.

Design rules (from AGENTS.md)

- **Code makes every decision.** LLMs only screen for manipulation, extract facts and write questions. `app/policy.py` is pure: no LLM, no I/O.
- **The input guard runs first** on every expense and every follow-up answer. A manipulation attempt is rejected.
- **Fail closed.** If any check can't complete, the result is `flag`, never `approve`.
- **Untrusted text goes into prompts only inside tags**, and every LLM call uses structured output.
- **One path.** Batch and portal share `pipeline.process`; what is stored is exactly what is POSTed.
- **Append-only.** Decisions and follow-ups are revisions. An answer adds data and triggers a full re-evaluation; it never sets the decision.
- No keys in logs or code; `.env` is never committed.

2. Running it

Task	Command
Set up	`python3 -m venv .venv && .venv/bin/pip install -r requirements.txt`
Unit tests (offline, LLMs stubbed)	`.venv/bin/python -m pytest -q`
Eval (live LLMs, labelled data)	`.venv/bin/python eval.py` · `--suite mock` for the 100 synthetic cases
Mock downstream endpoint	`python mock_server.py` (localhost:8000)
Batch run	`.venv/bin/python batch.py [file]`
Portal + dashboard	`.venv/bin/uvicorn app.server:app --port 8001`; rebuild the UI with `cd frontend && npm run build`

Environment: `DECISIONS_ENDPOINT`, `EXPENSE_DB`, `LLM_MODEL`, `GUARD_DECISIONS_MODEL`, `GUARD_FALLBACK_MODEL`, `GUARD_REVIEW_THRESHOLD`, `GUARD_REJECT_THRESHOLD`, `LLM_TIMEOUT_S`, `LLM_DISABLED`, `HIGH_AMOUNT_REVIEW`, `EXPENSE_TODAY`, plus API keys from `.env`.

3. Repository layout

```
app/
  config.py      policy tables, model ids, thresholds, env      guard.py     input guard (stage 1)
  normalize.py   amount + date parsing                           llm.py       all LLM calls
  policy.py      pure rules engine                               pipeline.py  the one decision path
  jobs.py        background decision workers                     outbox.py    reliable delivery
  poster.py      HTTP POST of one decision                       store.py     SQLite persistence
  server.py      FastAPI REST + SSE + SPA                        prompts/     4 system prompts (.md)
frontend/src/    React SPA: pages/employee, pages/finance, components/, lib/
data/            past_decisions, pending_expenses, expected_pending, mock_cases, approval_codes, notes
tests/           11 test files + conftest.py, helpers.py
batch.py  eval.py  mock_server.py  starter.py
docs/            ARCHITECTURE.md, DECISIONS.md, adr/0001-0007, PROGRESS.md, REQUIREMENTS.md
ASSUMPTIONS.md   every assumption, Inferred or Confirmed      FutureWork.md  receipts, auth, scale
```

4. Decision flow

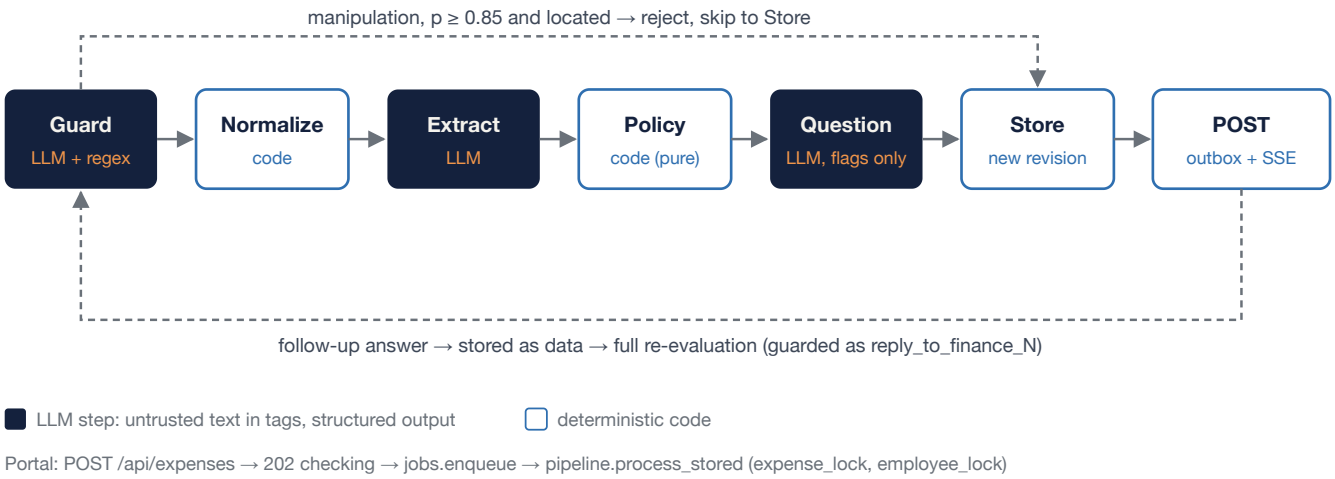


Figure 1. One expense through `pipeline.evaluate` (`app/pipeline.py:62`) and `_record_locked`.

#	Step	What happens	Where
0	Accept	Portal: the expense is upserted (version bumped on change), marked `checking`, a 202 is returned and a job is queued. Batch calls `process` directly.	`server.py:139`, `jobs.py:27`
1	Merge answers	Structured fields from follow-up answers (whitelisted) are merged over the raw expense; reply text becomes `reply_to_finance_N`.	`pipeline.py:69–73`
2	Guard	Unknown fields dropped, text sanitised. The screen scores every field and every reply. p ≥ 0.85 and located to a field → reject; 0.5–0.85 or unlocated → no auto-approve. Regex backstop when the screen finds nothing.	`guard.py:109`, `pipeline.py:76–88`
3	Normalize	Amounts must be plain dollars and cents; other currency symbols flag `non_usd`. Dates are ISO or US.	`normalize.py:58`
4	Extract	Claude maps category, client and city. An exact category is never re-mapped; an empty one is never filled in. If the LLM fails, a minimal fallback is used and the result is forced to flag later.	`llm.py:145`, `pipeline.py:93–118`
5	Policy	Pure rules return the decision and reasons (section 6).	`policy.py:53`
6	Post-policy checks	A suspect guard score turns approve into flag; with answers, the original submission is re-checked for duplicates.	`pipeline.py:124–137`
7	Fail closed	If the guard or LLM couldn't complete, the result is forced to flag.	`pipeline.py:139`
8	Human-final check	If Finance already decided this version, nothing is re-decided; after a later edit an approve becomes `edited_after_review`.	`pipeline.py:213–235`
9	Question	Flags only: the LLM drafts a question, with a template fallback.	`pipeline.py:181`
10	Store, notify, POST	A new revision with the exact payload; SSE notifies the UI; the outbox delivers it.	`pipeline.py:237`, `outbox.py:55`

5. Backend reference

Every module in `app/`, its constants and every function. Leading-underscore functions are private to their module.

> **`app/config.py`** 71 lines
>
> Central configuration: policy tables, model ids, endpoints. Every number the policy uses lives here, so the API's `/api/policy` hints and `policy.py` read the same values.

Name	What it is / does
`POLICY_VERSION`, `PROMPT_VERSION`	Stamped on every POSTed decision for audit (`2026-10-27.5`, `v3`).
`DECISIONS_ENDPOINT`, `DB_PATH`	Downstream URL (default `localhost:8000/decisions`) and SQLite file.
`LLM_MODEL`, `GUARD_DECISIONS_MODEL`, `GUARD_FALLBACK_MODEL`	`claude-haiku-5-5` for extraction and questions; `gpt-6-luna` for the guard; Claude as the guard fallback.
`GUARD_REVIEW_THRESHOLD`, `GUARD_REJECT_THRESHOLD`	0.5 and 0.85: the two bands on P(manipulation).
`LLM_TIMEOUT_S`, `LLM_DISABLED`	30 s timeout; `LLM_DISABLED=1` forces the fail-closed paths (tests).
`CATEGORIES`, `MEALS`, `UNCAPPED`	The 9 policy categories; Breakfast/Lunch/Dinner; Office, Software, GroundTransport, Airfare.
`MEAL_CAPS`, `HOTEL_CAP`, `HOTEL_CAP_TIER1`, `TIER1_CITIES`	$20 / $30 / $60; $200 a night, $350 in 8 Tier-1 cities (aliases such as "nyc", "sfo").
`RECEIPT_THRESHOLD`, `HIGH_AMOUNT_REVIEW`, `STALE_MONTHS`	Receipt needed over $25; anything over $1,000 goes to a human; older than 6 months is stale.
`today()`	The policy's "today"; `EXPENSE_TODAY` pins it so fixed-date data doesn't age.
`load_approval_codes()`	Reads the valid software approval codes from `data/approval_codes.json`.

> **`app/guard.py`** 143 lines
>
> Input guard, the first stage of the pipeline. Validates the schema, sanitises text and screens for manipulation.

Name	What it is / does
`ALLOWED_FIELDS`, `MAX_LEN`, `TEXT_FIELDS`	Whitelist of input fields with their types, length limits, and the free-text fields that get scanned.
`ZERO_WIDTH`, `CONTROL`, `HOMOGLYPHS`	Characters stripped or folded so hidden or look-alike text can't slip past the scan.
`BACKSTOP_PATTERNS`	The small regex list of reviewer-directed phrases (the last layer).
`sanitize_text(s)`	NFKC-normalises, removes zero-width characters, turns control characters into spaces, folds homoglyphs.
`_squash(s)`	Lower-cases and drops separators so "a.u.t.o-a p p r o v e" still matches.

`regex_scan(text)`	Returns the first matching evidence string, or None.
`check_schema(expense)`	Returns `(clean, issues)`: unknown keys (e.g. an injected `decision`) are dropped and reported; wrong types rejected; text truncated.
`scan_fields(fields)`	Runs `regex_scan` over each field; returns `[{field, evidence}]` hits.
`run_guard(expense, extra_text, llm_classifier)`	The full guard. Calls the classifier on all non-empty fields and replies; a confident, located hit goes to `manipulation` (reject), anything else positive to `suspect` (no auto-approve). A classifier exception sets `guard_error` (fail closed). Falls back to the regex scan when there is no hit.

`app/normalize.py` 80 lines

Deterministic parsing of the messy raw fields. Never guesses: anything ambiguous becomes an error the policy flags.

Name	What it is / does
`USD_MARKERS`, `AMOUNT_RE`	`$`, `US$`, `USD`; the strict dollars-and-cents pattern (thousands commas, at most 2 decimals).
`parse_amount(raw)`	Returns `(amount, foreign_symbol, error)`. Plain numbers and USD markers are USD; any other currency symbol (detected by Unicode category) is reported; "12,50", "1e1", "nan", "15.999" are errors.
`parse_date(raw)`	Returns `(date, error)`; accepts ISO and US MM/DD/YYYY.
`normalize(expense)`	Builds the normalised view (amount, foreign symbol, date, weekend, merchant, memo, receipt as literal `true` only, raw category); never raises, errors are collected.

`app/llm.py` 167 lines

All LLM calls. The model extracts facts and writes questions; it never decides. Every input is wrapped in a tag; every output is schema-checked.

Name	What it is / does
`LLMError`	The single error type every caller catches to fail closed.
`_prompt(name)`	Loads and caches a system prompt from `app/prompts/`.
`_openai()`, `_client()`	Cached OpenAI and Anthropic clients.
`_call_json(model, system, user, schema, max_tokens)`	One Claude call with JSON-schema output and server-side fallback. Raises `LLMError` on API errors, refusal, `max_tokens`, a missing text block or invalid JSON.
`_wrap(tag, obj)`	Serialises untrusted data as JSON inside `<tag>…</tag>`.
`GUARD_SCHEMA`, `EXTRACT_SCHEMA`, `QUESTION_SCHEMA`	The output schemas: injection/field/evidence; category, confidence, is_client, client_name, city; question.
`classify_injection_claude(fields)`	Fallback guard screen with Claude and `guard.md`.
`classify_injection_decisions(fields)`	Primary screen: OpenAI Decisions API with a predicate question (calibrated probability) and a choice question naming the field. No predicate answer → `LLMError`.

`classify_injection(fields)`	Decisions API, then Claude; both down → `LLMError` (the guard then fails closed).
`extract(expense)`	Sends whitelisted fields plus `followup_answers` to Claude with `extract.md`.
`generate_question(expense, reasons)`	Drafts one follow-up question from the expense and its gaps (`question.md`).

`app/policy.py` 197 lines

Pure, deterministic policy engine. Imports only `datetime`, `re` and `config`. The full rule list is in section 6.

Name	What it is / does
`_reason(rule, detail)`	Builds one `{rule, detail}` reason.
`PLACEHOLDER_MERCHANTS`, `MULTI_NIGHT_RE`	"N/A", "none", "TBD", "misc"…; the pattern for "3 nights", "2-night", "nights".
`is_placeholder_merchant(name)`	True for blanks, punctuation-only or placeholder names.
`months_before(day, months)`	Same day-of-month N months earlier, clamped (Aug 31 – 6 → Feb 28).
`is_tier1(city_text)`	True if the text names a Tier-1 city or alias.
`evaluate(norm, signals, history, approval_codes, today)`	The decision. Order: data flags → hard rejects (return reject) → data flags (return flag) → needs-a-human flags → approve with `within_policy`.
`_active(history, norm)`	Other expenses that could be paid: not this one, not rejected, with a USD amount and date.
`merchant_key(name)`	Letters only, so "Chipotle #12" equals "CHIPOTLE".
`detect_duplicate(norm, history)`	Same employee, merchant, date and amount within 1% as a live expense.
`detect_split(norm, category, is_client, history)`	Same employee, merchant and meal category within 1 day, each under the cap but combined over it.
`_approve_detail(…)`	The human-readable detail for an approval.

`app/pipeline.py` 312 lines

The one decision path shared by the batch run and the portal. `evaluate` is side-effect free; everything else stores, notifies and delivers.

Name	What it is / does
`expense_lock(expense_id)`	Per-expense lock: one decision at a time per expense.
`employee_lock(employee)`	Per-employee lock so parallel duplicates or splits from one person are seen.
`on_decision(fn)`, `notify(event)`	Register a listener (the server's SSE broadcast) and send it events.
`_exact_category(raw)`	Exact-match category mapping that wins over the LLM.
`_followup_texts(answers)`	The reply texts passed to the extractor as `followup_answers`.

`evaluate(raw, answers, history, use_llm, extract_fn, guard_fn)`	Classifies one expense (steps 1–7 in section 4). Returns decision, reasons, normalised view, signals, `suspected_manipulation`. `extract_fn`/`guard_fn` let tests and eval inject stubs.
`_template_question(raw, reasons)`	A per-rule fallback question when the LLM can't write one.
`build_question(raw, reasons, use_llm)`	LLM question, falling back to the template; returns `(question, source)`.
`process(raw, source, expense_id, external_id, use_llm)`	Full path for batch: store → evaluate → question → store decision → POST → notify.
`process_stored(expense_id, use_llm)`	Re-evaluates from the latest stored state (background jobs). None when Finance already decided this version.
`_decide_locked(expense_id, use_llm)`	Reads state once under the lock, applies the human-final rule, calls `evaluate` under the employee lock.
`_record_locked(…)`	Builds the payload, adds the question for flags, stores the revision, updates the normalised view, notifies.
`record_failure(expense_id, error)`	A crashed job becomes a stored, POSTed `check_failed` flag.
`Stale`	Raised when an override would apply to data the reviewer didn't see (→ 409).
`human_override(expense_id, decision, note, reviewer, expected_revision)`	Finance's decision, recorded and POSTed like any other with `decided_by="human"`. Refused if re-checking or the revision moved on.

`app/jobs.py` 86 lines

Background decision jobs for the portal.

Name	What it is / does
`enqueue(expense_id)`	Queues a decision; if one is already running for that id it is marked to re-run once (coalescing rapid edits).
`_run(expense_id)`	Calls `process_stored`; on an unexpected exception records `check_failed`, or status `error` if even that fails; re-runs if marked.
`recover()`	On startup, re-queues anything left `checking` or `error`.
`wait_idle(timeout)`	Blocks until no job is queued or running (tests, scripts).

`app/outbox.py` 111 lines

Reliable delivery of decisions downstream. 8 attempts, backoff from 5 s up to 300 s, polled every 2 s.

Name	What it is / does
`_attempt(expense_id, revision, now)`	Claims the row atomically, POSTs the stored payload once, records sent / failed (with the next attempt time) / dead.
`deliver(expense_id, revision, payload)`	The first attempt, right after the decision is stored.

`run_due(now, ignore_backoff)`	Retries everything that is due; returns how many were sent.
`flush()`	One immediate pass over every undelivered decision (end of a batch).
`retry(expense_id, revision)`	Finance's "Retry now": one immediate attempt, latest revision only; a dead row goes back to dead if it fails.
`retry_all()`	Revives every dead latest decision and tries all undelivered now.
`_loop()`, `start()`, `stop()`	The background retry thread and its lifecycle (started by the server).

`app/poster.py` 26 lines

HTTP delivery of one decision.

Name	What it is / does
`post_decision(decision)`	POSTs one decision with `Idempotency-Key` = `decision_key` (`{id}:{revision}`), 5 s timeout; True on HTTP 200. On failure logs only the decision key and the error.

`app/store.py` 484 lines

SQLite persistence (WAL). Decisions, versions and follow-ups are append-only history. Schema in section 8.

Name	What it is / does
`SCHEMA`, `INDEXES`, `MIGRATIONS`	Table definitions, indexes, and additive migrations for older DB files.
`now()`, `conn()`, `reset(path)`, `_row(r)`	UTC timestamp; per-thread connection; point at another DB (tests); row → dict with JSON decoded.
`next_portal_id()`	Allocates the next `WEB-####` id.
`_record_version(…)`	Appends a row to `expense_versions`.
`add_portal_expenses(rows)`	Inserts an upload in one transaction, already `checking`; all or nothing.
`idempotent_ids(key)`, `remember_idempotent(key, ids)`	Look up and record the result of a keyed request.
`upsert_expense(id, raw, source, external_id)`	Insert or replace the submitted fields; bumps the version on change.
`set_normalized`, `set_status`, `ids_with_status`	Update the normalised view and status; list ids in a status (recovery).
`add_answer(expense_id, answer)`	Appends an answer and marks the open follow-up answered.
`add_decision(…)`	Allocates the next revision and inserts it atomically; sets `payload["revision"]` and the decision key.
`record_post`, `claim_post`, `post_status`	Record a delivery outcome; atomically claim a deliverable row (latest revision only, so the inline attempt and the worker never both send it); read the status.
`delivery_problems`, `revive_post`, `reset_stale_sending`, `due_posts`	Failed/dead latest decisions; make dead rows deliverable; reset rows left `sending` by a crash; rows due for retry, oldest first.

`latest_decision`, `get_expense`, `expense_detail`	Latest revision; current expense; expense with all decisions, follow-ups and versions.
`employee_history(employee, exclude_id)`	Normalised view of the employee's other expenses (duplicate and split checks).
`query_expenses(f, sort, direction, limit, offset)`	Filtered, sorted page with the latest decision; filters: status, manipulation, source, employee, category, dates, amounts, rule, delivery, text. Sorts are whitelisted.
`list_expenses`, `employees`, `stats`	Simple lists, and aggregates for the Finance overview from the latest decision only.

> **`app/server.py`** 421 lines
>
> FastAPI app: REST API, SSE stream, CSV export, and the built React SPA. Endpoints are listed in section 7.

Name	What it is / does
`lifespan(_app)`	Startup: open the store, start the outbox (which resets stale `sending` rows), re-queue unfinished jobs; shutdown stops the outbox.
`_offer`, `_broadcast`, `stream`	`_broadcast` is registered with `pipeline.on_decision` at import. Fans events out to per-client queues (max 500; a client that falls behind is cut off and reconnects); `/api/stream` serves SSE.
`_spa`, `spa`	Serve `frontend/dist/index.html` for client-side routes.
`ExpenseIn`, `OverrideIn`	Request bodies for create/edit and Finance override.
`_raw`, `_accept`, `_submit`	Build the raw expense; store it as `checking` and enqueue; create one portal expense.
`_once(request, scope, work)`	Runs a create/upload/answer at most once per `Idempotency-Key` and endpoint.
`_filters`, `_cell`, `_export_row`	Query params → store filters; formula-safe CSV cell (prefixes `= + - @`, tab, CR); one export row.
`_parse_upload(name, content)`	Parses a JSON or CSV upload; any row that isn't an expense object refuses the whole file.
`_answer(…)`	Validates there is an open question, saves an attached receipt, stores the answer, re-queues the decision.
Route handlers	`get_stats`, `get_policy`, `list_deliveries`, `retry_all_deliveries`, `retry_delivery`, `get_employees`, `list_expenses`, `export_expenses`, `get_expense`, `create_expense`, `edit_expense`, `upload_expenses`, `answer_followup`, `override`.

6. Policy rules reference

Rules as `policy.evaluate` applies them, plus the ones the pipeline adds. A reject from trusted fields outranks a flag.

Rule id	Condition	Outcome
Data that can't be evaluated		
`missing_category` / `unknown_category`	No category, or the LLM couldn't map it with confidence ≥ 0.7	flag
`missing_employee` / `missing_merchant`	Blank employee; blank or placeholder merchant	flag
`non_usd` / `invalid_amount` / `non_positive_amount`	Another currency symbol; unreadable amount; zero or negative	flag
`invalid_date` / `future_date` / `stale_date`	Missing or unreadable; after today; more than 6 months ago	flag
Hard rejects (only when the amount and category are usable)		
`weekend_meal`	Meal on a Saturday or Sunday with no client	reject
`meal_over_cap`	Meal over $20 / $30 / $60 with no client	reject
`hotel_over_cap`	One night (or unstated) over $200, or $350 in a Tier-1 city	reject
`multi_night_hotel`	Hotel memo or merchant mentions more than one night	flag
`entertainment_not_client`	Entertainment with no named client	reject
Needs a human		
`missing_receipt`	Amount over $25 and `receipt` isn't literal true	flag
`missing_approval_code` / `unknown_approval_code`	Software with no code, or a code not in `approval_codes.json`	flag
`possible_duplicate`	Same employee, merchant, date and amount within 1% as a live expense	flag
`possible_split`	Same employee, merchant and meal within a day, combined over the cap	flag
`high_amount_review`	A would-be approve over $1,000	flag
Added by the pipeline		
`suspected_manipulation`	Guard: p ≥ 0.85 and located, or a regex hit	reject
`possible_manipulation`	Guard score 0.5–0.85 or unlocated, on a would-be approve	flag
`llm_unavailable` / `guard_unavailable`	Extraction or guard couldn't complete	flag
`edited_after_review`	Edited after Finance decided, and would auto-approve	flag
`check_failed`	The decision job crashed	flag
`human_review`	Finance override, with the reviewer's note	as decided
`within_policy`	Nothing above applied	approve

7. REST and SSE API

Endpoint	Purpose
`GET /api/stream`	SSE: queued, processing, decision, error, delivery events
`GET /api/stats`	Overview aggregates (latest decision only)
`GET /api/policy`	Policy numbers for the form's hints; approval codes deliberately left out
`GET /api/employees`	Known employees (identity picker)
`GET /api/expenses`	Filtered, sorted, paged list; `X-Total-Count` header
`GET /api/expenses/export.csv`	Same filters, every page, formula-safe cells
`GET /api/expenses/{id}`	Detail: decisions, follow-ups, versions
`POST /api/expenses`	Create (202, decided in the background); idempotent
`PUT /api/expenses/{id}`	Edit; new version, re-decided
`POST /api/expenses/upload`	JSON/CSV upload, atomic; ids and labels in the file ignored; idempotent
`POST /api/expenses/{id}/answer`	Reply to the open question (text, code, category, amount, merchant, receipt file); 409 if none open; idempotent
`POST /api/expenses/{id}/override`	Finance decision; 409 if stale or re-checking; 400 reversing an auto decision without a note
`GET /api/deliveries`	Failed and dead latest deliveries
`POST /api/deliveries/{id}/{rev}/retry`, `/retry-all`	Retry one, or all
`GET /`, `/employee/*`, `/finance/*`	The React SPA

8. Data model and what is POSTed

Table	Columns that matter
`expenses`	`id` (EXP-… or WEB-####), `employee`, `source` (batch | portal), `external_id`, `raw_json` (as received, pre-guard), `normalized_json`, `answers_json`, `version`, `status`, timestamps
`expense_versions`	Every submitted version, append-only (ADR 0007): version, raw, created_at
`decisions`	One row per evaluation: `revision`, `expense_version`, `decision`, `decided_by`, `payload_json` (exact POST body), signals, post status (pending, sending, sent, failed, dead, superseded), attempts, next attempt, error
`followups`	Question, answer, status (open, answered, superseded)
`idempotency`	Request key (scoped to its endpoint) → resulting expense ids

What is POSTed (stored byte-for-byte in `payload_json`): id, employee, decision, reasons, question, amount (USD or null), category, category_mapped_from, client, suspected_manipulation, revision, source, decided_by, policy_version, prompt_version, decided_at, decision_key. Downstream keeps the latest revision; older undelivered ones become `superseded`.

Signals (stored, not POSTed): guard result and screen scores, the extraction output, LLM errors, question source, job errors. These are the audit trail for why a decision was made.

9. Frontend reference

React 18 SPA built with Vite and Tailwind, served by FastAPI from `frontend/dist`. No auth: the employee picks their identity (ASSUMPTIONS A4).

Routes and pages

Route · file	What it does
`/` · `pages/Landing.tsx`	Choose Employee or Finance.
`/employee` · `employee/EmployeeHome.tsx`	The employee's expenses (`Mine`) with status and open questions.
`/employee/new` · `employee/NewExpense.tsx`	Form with live policy hints, or a CSV/JSON upload with preview (`Upload`); one idempotency key per visit.
`/employee/expenses/:id` · `employee/EmployeeExpense.tsx`	Decision stamp, the question, the answer form, edit, timeline.
`/finance` · `finance/Overview.tsx`	Stats, decision mix, spend by category.
`/finance/queue` · `finance/Queue.tsx`	Ready for a decision vs waiting on the employee.
`/finance/expenses` · `finance/AllExpenses.tsx`	Filtered, paged (50) list; filter chips, saved views, CSV export; `apiParams` maps the URL to API params.
`/finance/expenses/:id` · `finance/FinanceExpense.tsx`	`WhatChanged` (edits and replies between the last two decisions), `ReadAs` (how the system read it), `Checks` (guard and signals), `Decide` (override with confirm; note required to reverse).

`/finance/delivery` · `finance/Delivery.tsx`	Failed and dead deliveries with retry.

Components

File	What it does
`App.tsx`, `main.tsx`	Router and app root.
`EmployeeLayout`, `FinanceLayout`	Shell and navigation for each side.
`ExpenseForm`	Create/edit form: `validate`, `toInput`, `clean` (plain numbers sent as numbers, anything else as typed for the server to judge); drafts.
`AnswerForm`	Reply to Finance; shows only the fields the open gaps need.
`Timeline`	Each revision with the question asked and the answer that led to the next.
`Charts`	`DecisionMix`, `SpendByCategory`, `Legend`; colours always paired with labels.
`Stamp`, `StatusPill`	Decision as a stamp; compact status (icon + label).
`IdentityPicker`, `ThemeToggle`, `LiveDot`, `Toast`	Who am I; light/dark; SSE connection state; notifications.
`States`, `ErrorBoundary`	Loading, error, empty states; keeps a render bug in one page from blanking the app.

Library (`src/lib`)

File	What it does
`api.ts`	Typed fetch wrapper for every endpoint; sends `Idempotency-Key`.
`types.ts`	Types for the payload (exactly what is POSTed), expenses, decisions, follow-ups, policy.
`live.ts`, `useData.ts`	One shared EventSource; `useLiveEvents`/`useLiveStatus`; data hook that refetches on relevant events, coalesces bursts and resyncs after reconnect or tab focus.
`policyHints.ts`	`hintsFor`: advisory hints from `/api/policy` (caps, receipt, weekend, foreign amount, stale date); never blocks a submit.
`changes.ts`	`changesSincePrevious`: field edits and replies between the last two decisions.
`review.ts`	`readyForDecision` / `waitingOnEmployee`: splits the queue; some rules only a person can resolve.
`draft.ts`	Unsent input kept in sessionStorage; warns before closing; `newActionKey`.
`format.ts`, `csv.ts`, `identity.ts`, `views.ts`	Money, dates, labels; CSV parser for the upload preview; stored identity; saved Finance views.

10. Scripts and prompts

File	What it does
`batch.py · main()`	Reads a file (default `data/pending_expenses.json`), drops any `decision` label, runs each in order (split detection reads earlier ones) through `pipeline.process(source="batch")`, prints decisions, then `outbox.flush()`.
`eval.py`	Scores the decision logic, no storage or POSTs. `run` evaluates a suite; `report` prints accuracy and misses; `build_history` builds prior expenses offline; `rule_matches` checks the expected rule; `run_mock` runs the 100 mock cases; `main` picks suites.
`mock_server.py · Handler`	The provided downstream endpoint on :8000: `do_POST` accepts decisions and logs them; `do_GET` serves `/health` and `/decisions` (all received).
`starter.py`	The exercise's original scaffold (TODO stubs); kept for reference, not used by the app.

Prompt ( )	Used by · purpose
`guard_decisions.md`	`classify_injection_decisions`: is any field text aimed at the reviewer? Data in `<fields>`.
`guard.md`	`classify_injection_claude`: same screen for Claude, returns field and evidence.
`extract.md`	`extract`: facts for the policy engine; explicitly "you do NOT decide".
`question.md`	`generate_question`: one short message covering every gap in `<gaps>`.

11. Tests

Run offline: `conftest.py` sets `LLM_DISABLED=1` and LLMs are stubbed. `helpers.expense(**kw)` builds a valid base expense. 112 test functions; 156 cases with parametrisation.

File	Tests	Covers
`test_policy.py`	24	Every rule: caps, weekend, client, hotel tiers, multi-night, receipt, codes, split, duplicate, high amount, dates, placeholders
`test_server.py`	24	API round trips: flag → answer → approve, uploads, edits, overrides (stale, re-checking), policy endpoint, deliveries, idempotency
`test_adversarial.py`	14	Injection in fields and replies, injected `decision` key, LLM and guard down → flag
`test_guard.py`	11	Schema, sanitising, screen primary, regex backstop, bands, unlocated hits, screen failure
`test_outbox.py`	10	Retry until sent, dead, flush, idempotency header, retry once, supersede, stats
`test_races.py`	7	Superseded never delivered, human-final, edit after review, parallel duplicates, answer can't clear duplicate, crash → flag
`test_normalize.py`	6	Amount and date parsing, currencies, malformed amounts
`test_query.py`	6	List filters, sorts, paging
`test_mock_cases.py`	5	The synthetic failure-mode cases, offline
`test_versions.py`	3	Expense version history

`test_concurrency.py`	2	Unique revisions under concurrent overrides and re-processing

12. Safety boundaries

Each rule in AGENTS.md, the line that enforces it and the test that covers it. **GAP** marks a guarantee the code enforces but no test pins.

Rule	Enforced at	Test
Code decides; policy is pure	`policy.py:15–18` (imports only dt, re, config); `pipeline.py:121`	`test_policy.py`; purity: **GAP**
Guard first, on expenses and replies	`pipeline.py:76` (replies at :73); `guard.py:109`	`test_guard.py:49, 59–70`; `test_adversarial.py:24`
Manipulation → reject	`pipeline.py:82`	`test_adversarial.py:19`; `test_server.py:118`
Fail closed	`pipeline.py:139`; `pipeline.py:268` (`check_failed`)	`test_adversarial.py:52, 59`; `test_races.py:94`
Untrusted text in tags, structured output	`llm.py:66 _wrap`; `:49` json_schema; `:93` typed answers	**GAP**
One path; stored = POSTed	`batch.py:26`; `outbox.py:35` reads `payload_json`	`test_server.py:108`; shared path: **GAP**
An answer adds data, never sets the decision	`server.py:381`; `pipeline.py:71` (ALLOWED_FIELDS), `:132` (duplicate)	`test_adversarial.py:29`; `test_races.py:84`
Human decision final per version	`pipeline.py:219, 229, 284` (Stale → 409)	`test_races.py:46, 54`; `test_server.py:173, 189`
No keys in logs or code	Keys only from env (`config.py`); `.env` in `.gitignore`	**GAP**

Other defences: CSV export cells are formula-safe (`server.py:250`); approval codes are never sent to the browser (`server.py:183`); uploads ignore ids and labels in the file.

13. LLM providers and fallback

- **Guard:** OpenAI Decisions API (`gpt-6-luna`) → Claude screen if it fails → regex backstop whenever there is no hit. Both screens failing sets `guard_error`, which becomes a flag. Calibration in config: benign ≤ 0.05, attacks 1.00, one borderline reply 0.01–0.70 across runs, hence the review band.
- **Extraction and questions:** Claude (`claude-haiku-5-5`) with JSON-schema output and server-side fallback; questions fall back to a per-rule template.
- **Only the guard judges manipulation.** The extractor's old injection flag was removed after it rejected a benign reply (ASSUMPTIONS S5).

14. Failure modes and concurrency

Failure	Result
Guard or extraction unavailable	Flag `guard_unavailable` / `llm_unavailable`; never approve
Decision job crashes	Flag `check_failed`, stored and POSTed; if that fails too, status `error`

Downstream endpoint down	8 attempts with backoff (5 s → 300 s), then `dead`; Finance can retry the latest once
Process restart	`checking`/`error` re-queued; stale `sending` rows reset
Finance overrides a stale view	409; the page reloads
A client retries a request	Same `Idempotency-Key` → the first result, nothing duplicated
SSE client falls behind	Cut off after 500 queued events; it reconnects and refetches

Concurrency: `expense_lock` serialises decisions per expense; `employee_lock` makes one person's parallel expenses see each other (duplicates, splits); `add_decision` allocates revisions atomically; `claim_post` ensures one sender per row; jobs coalesce rapid edits into one re-run.

15. Decisions (ADRs)

ADR	Decision	Rejected alternative
0001	Code decides; LLM only extracts and writes questions	LLM outputs the decision
0002	Guard first: Decisions API primary, Claude fallback, regex backstop	Regex only; OpenAI Moderation API
0003	Manipulation is rejected; unknowns and failures are flagged	Flagging manipulation for review
0004	Everything is USD; other currency symbols are flagged	Static FX table; currency field
0005	SQLite with append-only decision and follow-up history	JSON file; in-memory store
0006	Backstops: duplicates, >$1,000, future or stale dates, typo amounts, multi-night hotels	Reject duplicates; no ceiling
0007	Keep every submitted version of an expense	Diffing extracted signals; per-edit diffs

16. Known gaps and planned work

- Tests for the gaps in section 12: policy purity, prompt shape, the shared batch/portal path, keys in logs
- Browser check of the v3 screens; frontend tests (Vitest, a new dependency that needs approval)
- Per-person caps and an alcohol rule (open questions with Finance)
- Auth and roles, receipt OCR, an approval-code service (see FutureWork.md)